HESTIM — Engineering & Business School

Projet PACTE — 4e année

Année universitaire 2026–2027

AutoMaze AI

Classification d'objets routiers par Machine Learning
pour les véhicules autonomes

Livrable 1 : dossier Go/noGo

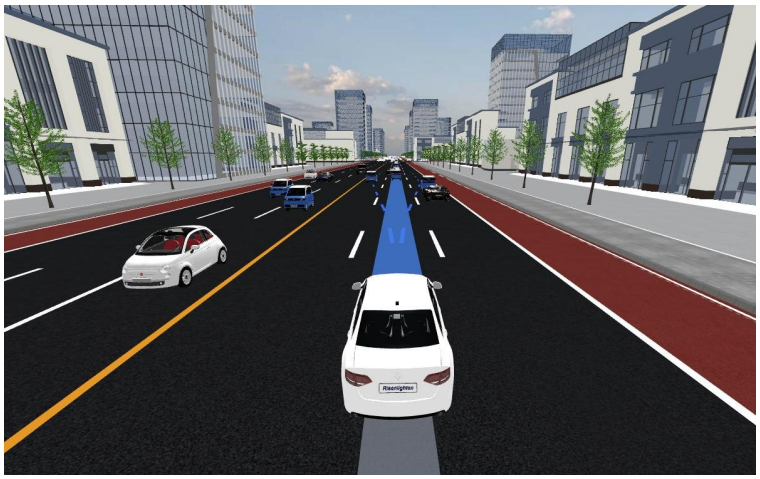


Illustration d'un scénario de conduite (source externe, annexe A)

Équipe : Rodrigue Nzuzi
[Membre 2]
[Membre 3]

Encadrant : [Nom de l'encadrant]

Date : 6 novembre 2026

Table des matières

Introduction générale **3**

1 Présentation du projet et cahier des charges **4**

1.1 Présentation et contexte 4

1.2 Problématique 4

1.3 Objectif général et objectifs spécifiques 4

1.4 Périmètre fonctionnel 5

1.5 Besoins fonctionnels 6

1.6 Besoins non fonctionnels 7

1.7 Contraintes techniques et académiques 8

1.8 Méthodologie et planning prévisionnel 8

2 Analyse fonctionnelle et conception du système **10**

2.1 Identification des acteurs 10

2.2 Description des cas d'utilisation 10

2.3 Scénarios de fonctionnement 11

2.4 Diagramme de cas d'utilisation 12

2.5 Diagramme d'activité 12

2.6 Diagramme de séquence 13

2.7 Diagramme de composants 14

2.8 Description des interactions entre modules 15

3 Étude et préparation des données **16**

3.1 Étude des datasets disponibles 16

3.2 Comparaison et justification du dataset retenu 16

3.3 Présentation des classes 16

3.4 Analyse quantitative des données 17

3.5 Analyse qualitative des images 17

3.6 Nettoyage des données 18

3.7 Redimensionnement et normalisation 18

3.8 Séparation entraînement, validation et test 19

4 Architecture technique et conception des modules **20**

4.1 Architecture globale d'AutoMaze AI 20
4.2 Environnement de simulation 21
4.3 Acquisition des images par caméra virtuelle 22
4.4 Module de classification par Machine Learning 23
4.5 Module de localisation et association des objets 23
4.6 Représentation de l'environnement et des obstacles 24
4.7 Algorithme de planification Dijkstra 24
4.8 Système de décision 24
4.9 Contrôle du véhicule virtuel 25
4.10 Communication et intégration des modules 25

5 Méthodologie expérimentale et évaluation 26
5.1 Extraction des caractéristiques 26
5.2 Présentation des algorithmes SVM, Random Forest et k-NN 26
5.3 Protocole expérimental 27
5.4 Métriques d'évaluation 27
5.5 Méthode de comparaison des modèles 27
5.6 Critères de sélection du modèle final 28
5.7 Protocole de validation dans le simulateur 28

6 Étude de faisabilité et validation Go/noGo 29
6.1 Faisabilité technique 29
6.2 Faisabilité matérielle et logicielle 29
6.3 Identification des risques 30
6.4 Preuves de faisabilité disponibles 30
6.5 Planning de réalisation 31
6.6 Critères de décision Go/noGo 32

Conclusion générale 33

Bibliographie 34

A Sources des illustrations 35

B Usage de l'intelligence artificielle 36

Introduction générale

Contexte. Un véhicule autonome doit percevoir son environnement, décider puis agir, sans conducteur pour corriger ses erreurs. Pour cela, il doit d'abord comprendre ce qui se trouve devant lui. Un piéton qui traverse appelle un arrêt, alors qu'un camion à l'arrêt se contourne. Une erreur de reconnaissance peut donc conduire à une mauvaise décision.

La perception routière. La perception d'un véhicule autonome comporte plusieurs tâches : trouver les objets dans l'image (la détection), dire de quoi il s'agit (la classification), puis estimer leur position et leur mouvement. Notre projet PACTE de 4e année porte sur la classification : à partir de l'image d'un objet, reconnaître de quel usager de la route il s'agit.

Problématique. Nous chercherons à savoir si un modèle entraîné de zéro sur un jeu de données de taille limitée peut reconnaître les usagers de la route assez bien pour guider les décisions d'un véhicule virtuel. Le chapitre 1 détaille cette question.

Objectifs. Le cœur du projet reste celui du sujet : constituer un jeu de données, extraire des caractéristiques, comparer SVM, Random Forest et k-NN, analyser les classes difficiles, puis prédire la classe de nouvelles images. Nous y ajoutons une démonstration : un véhicule virtuel qui utilise nos prédictions pour rejoindre sa destination dans les rues de Casablanca.

Démarche. Ce dossier suit la logique du projet :

— le chapitre 1 présente le projet, son périmètre et son cahier des charges ;

— le chapitre 2 décrit le fonctionnement attendu du système à l'aide de diagrammes ;

— le chapitre 3 étudie et prépare les données ;

— le chapitre 4 détaille l'architecture technique de chaque module ;

— le chapitre 5 fixe la méthode d'expérimentation et d'évaluation ;

— le chapitre 6 rassemble les preuves de faisabilité et les critères de la décision Go/noGo.

Ce dossier prépare la décision de Go/noGo, qui compte pour 15 % de la note finale.

Chapitre 1

Présentation du projet et cahier des charges

1.1 Présentation et contexte

AutoMaze AI reconnaît six usagers de la route à partir de l'image d'un objet : personne, vélo, voiture, moto, bus et camion. Le modèle renvoie une classe et un score de confiance. Une fois entraîné, il est testé sur des images qu'il n'a jamais vues, puis utilisé par un véhicule virtuel qui roule dans les vraies rues de Casablanca.

Nous sommes trois étudiants, et chacun de nous est responsable d'un rôle :

Rôle	Responsable	Missions
Données	[Nom]	Exploration, prétraitement, recadrages et découpage
Modèles	[Nom]	Extraction des caractéristiques, classifieurs, journal des essais
Évaluation et intégration	[Nom]	Évaluation finale, analyse des erreurs, export du modèle, lien avec le simulateur

1.2 Problématique

> *Un modèle entraîné de zéro sur un jeu de données de taille limitée peut-il reconnaître les usagers de la route assez bien pour guider les décisions d'un véhicule autonome virtuel ?*

Trois questions en découlent :

— Parmi SVM, Random Forest et k-NN, quel classifieur obtient le meilleur score sur les mêmes caractéristiques ?

— Quelles classes sont les plus difficiles à reconnaître, et pourquoi ?

— Que fait le véhicule quand le modèle se trompe, en particulier quand il ne reconnaît pas un piéton ?

1.3 Objectif général et objectifs spécifiques

Objectif général. Développer et évaluer un classifieur d'objets routiers entraîné de zéro, puis montrer comment ses prédictions guident un véhicule virtuel.

N°	Objectif spécifique	Mesure de réussite
OS1	Décrire le jeu de données retenu	Provenance, volume, répartition par classe, résolution et qualité documentées
OS2	Préparer les images	Nettoyage, recadrage, normalisation et découpage sans fuite de données
OS3	Comparer SVM, Random Forest et k-NN	Accuracy, précision, rappel, F1 et matrice de confusion, sur les mêmes données
OS4	Analyser les classes difficiles	Paires de classes confondues et images mal classées expliquées
OS5	Prédire sur de nouvelles images	Affichage du type « Image → Voiture (92 %) »
OS6	Intégrer le modèle au véhicule virtuel	Chaque prédiction déclenche une décision, et les erreurs graves sont comptées

Pour juger nos modèles, nous retenons surtout le F1 macro, car il donne le même poids à chaque classe. Nous ne fixons pas de valeur cible à l'avance : la référence sera le meilleur modèle classique, comparé à des scores planchers (section 5.4).

1.4 Périmètre fonctionnel

Le travail de Machine Learning est le cœur scientifique du projet. L'intégration sert à montrer son utilité dans un véhicule autonome virtuel, sans devenir le sujet principal.

Module	Responsabilité	Importance
Dataset	Fournir les images des objets routiers	Essentielle
Prétraitement	Préparer les images pour l'apprentissage	Essentielle
Classification ML	Identifier la classe d'un objet	Essentielle
Évaluation	Comparer les classifieurs	Essentielle
Simulateur	Fournir un environnement de démonstration	Intégration
Localisation	Fournir les coordonnées des objets	Intégration
Planification	Calculer le chemin le plus court	Intégration
Décision	Choisir le comportement face aux obstacles	Intégration
Contrôle	Commander le véhicule virtuel	Intégration

Hors périmètre : la détection d'objets. Notre modèle ne cherche pas où se trouvent les objets dans une grande image : ce serait le rôle d'un détecteur comme YOLO ou R-CNN (figure 1.1), interdit par les règles du cours. Il reçoit l'image d'un seul objet et dit de quoi il s'agit. Dans le simulateur, c'est la position connue de l'objet qui remplace la détection. La vidéo et la physique réaliste du véhicule restent aussi hors périmètre.

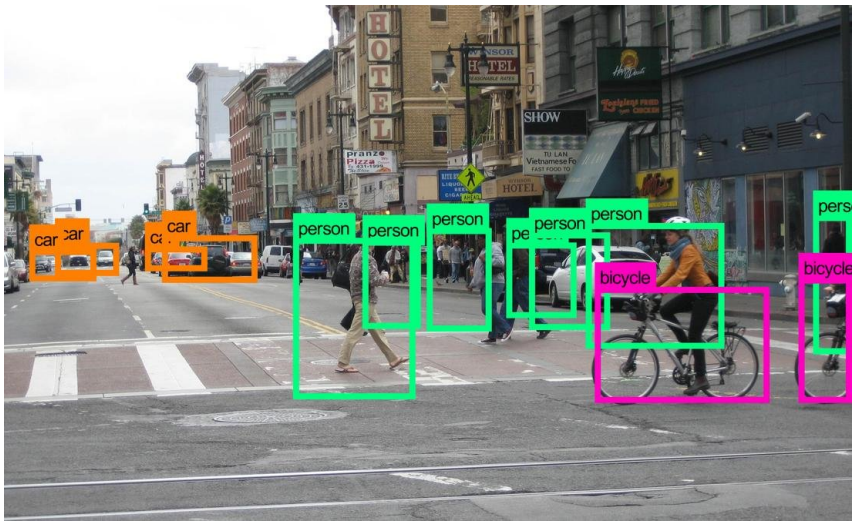


Figure 1.1 – Détection d'objets avec un réseau à régions (R-CNN) : chaque objet est localisé par une boîte. C'est ce que notre projet *ne fait pas* : nous classons un objet déjà isolé. Source externe (annexe A).

1.5 Besoins fonctionnels

Les exigences EF01 à EF06 reprennent le sujet. Les exigences EF07 à EF14 relèvent de l'intégration.

ID	Exigence	Critère d'acceptation
EF01	Charger et explorer le dataset routier	Effectifs par classe, tailles et exemples présentés (notebook 01)
EF02	Prétraiter les images	Images filtrées, recadrées, redimensionnées et normalisées ; découpage sans photo partagée (notebook 02)
EF03	Extraire les caractéristiques visuelles	Descripteurs HOG calculés pour toutes les images (notebook 03)
EF04	Entraîner plusieurs modèles de classification	SVM, Random Forest et k-NN entraînés sur les mêmes caractéristiques
EF05	Évaluer et comparer les modèles	Tableau commun : accuracy, précision, rappel, F1 et matrices de confusion
EF06	Prédire la classe d'un nouvel objet	Une image inconnue donne une classe et un score affichés
EF07	Récupérer les coordonnées des objets dans le simulateur	Chaque obstacle a un tronçon de rue et une position sur ce tronçon
EF08	Associer les prédictions aux objets virtuels	Chaque prédiction est rattachée à l'obstacle dont provient l'image
EF09	Représenter les obstacles dans l'environnement	Tronçons bloqués ou pénalisés selon la décision
EF10	Calculer le chemin vers la destination avec Dijkstra	Plus court chemin trouvé sur la carte, vérifié par des tests
EF11	Adapter le trajet en présence d'un obstacle	Nouveau calcul après chaque obstacle bloquant
EF12	Décider de ralentir, s'arrêter ou contourner	Une action par classe prédite, et un arrêt prudent si le score est trop faible
EF13	Commander le véhicule virtuel	Le véhicule avance, ralentit, s'arrête, fait demi-tour ou change de rue
EF14	Afficher les prédictions et les décisions du véhicule	Image, classe, score et action visibles à l'écran

1.6 Besoins non fonctionnels

ID	Exigence	Critère d'acceptation
ENF1	Entraînement de zéro	Aucun YOLO, détecteur ou poids préentraîné dans notre modèle
ENF2	Évaluation fiable	Découpage par photo d'origine ; jeu de test utilisé une seule fois, pour l'évaluation finale
ENF3	Reproductibilité	Graines fixées, essais notés, notebooks exécutables de bout en bout
ENF4	Cohérence du prétraitement	Même taille, même ordre des couleurs et même normalisation dans les notebooks et le simulateur
ENF5	Séparation vérité terrain / perception	Les décisions du véhicule n'utilisent jamais la vraie classe connue du simulateur
ENF6	Honnêteté des résultats	Aucune métrique inventée ; sans modèle, l'interface affiche « SIMULATION — modèle réel non chargé »
ENF7	Licences respectées	COCO, OpenStreetMap (ODbL) et illustrations citées
ENF8	Temps de réponse	Une prédiction en quelques millisecondes sur CPU, pour une démonstration fluide

1.7 Contraintes techniques et académiques

— **Règles du cours :** pas de YOLO, pas de détecteur et pas de poids préentraînés pour notre modèle. Les classifieurs à comparer sont ceux du sujet : SVM, Random Forest et k-NN.

— **Calendrier :** séance 1 le 4 novembre, séance 2 le 25 novembre, soutenance le 16 décembre 2026. Les séances ne se décalent pas.

— **Matériel :** aucun membre n'a de GPU. Les traitements doivent tenir sur un CPU ou sur le GPU gratuit de Google Colab.

— **Budget :** nul. Tous les outils et toutes les données retenus sont gratuits.

— **Licences :** les images COCO gardent leur licence Flickr ; elles ne sont donc pas redistribuées dans le dépôt Git.

Ressource	**Détail**	**Solution**
Données	Images des 6 classes, étiquettes fiables, licence compatible	COCO 2017, à valider par l'encadrant
Calcul	Entraînement des modèles	CPU local, Google Colab en appui
Stockage	Données, poids et figures partagés	Dossier Google Drive partagé
Poste local	Fenêtre du simulateur, que Colab ne peut pas ouvrir	Disponible
Encadrement	Validation du dataset et du périmètre	Section 6.6

1.8 Méthodologie et planning prévisionnel

Le projet suit deux phases (figure 1.2). Dans la première, nous développons et entraînons le modèle dans des notebooks versionnés sur GitHub. Dans la seconde, le modèle exporté est chargé par le simulateur, sans jamais y être réentraîné.

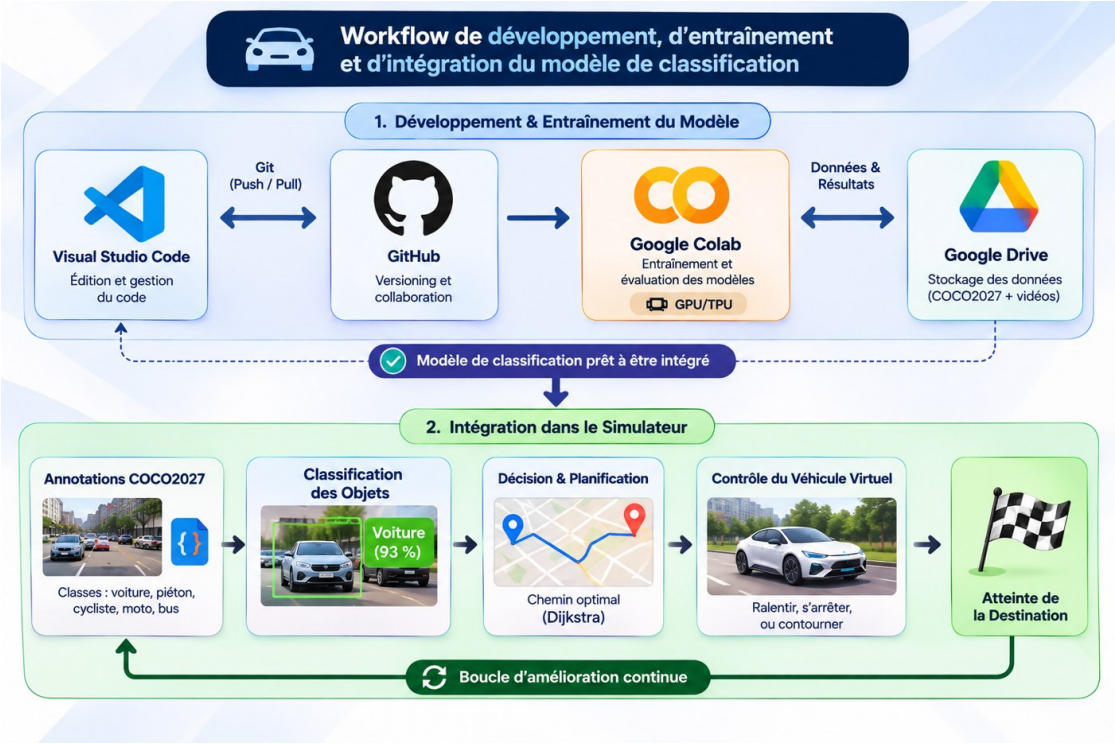


Figure 1.2 – Flux de développement et d'intégration. Deux corrections s'imposent : il faut lire COCO 2017 (et non « COCO2027 »), et nos classes comprennent aussi le camion. Le modèle classe des objets déjà isolés, sans boîte de détection.

1. **Données (notebooks 01 et 02) :** acquisition, exploration, nettoyage, recadrage et découpage.
2. **Modèles (notebook 03) :** extraction HOG, puis entraînement et réglage de SVM, Random Forest et k-NN sur la validation.
3. **Évaluation (notebook 05) :** mesure unique sur le jeu de test, analyse des erreurs, prédiction sur de nouvelles images.
4. **Intégration :** export du modèle, branchement dans le simulateur, puis comparaison sur plusieurs épisodes.

Organisation. Une seule personne modifie un notebook à la fois, et chaque notebook est relu par un autre membre avant chaque séance. Les décisions importantes sont notées dans un journal. Le planning détaillé figure en section 6.5.

Chapitre 2

Analyse fonctionnelle et conception du système

2.1 Identification des acteurs

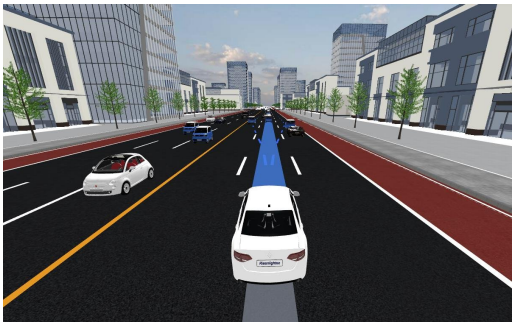
Acteur	Type	Rôle
Équipe projet	Humain	Prépare les données, entraîne et compare les modèles, évalue le modèle final
Utilisateur de la démo	Humain	Classe une image ou lance un épisode de conduite
Encadrant	Humain	Valide les choix (dataset, périmètre) et suit l'avancement
Jury	Humain	Juge les résultats, le rapport et la démonstration
Simulateur	Système	Fournit l'environnement, les positions des objets et leurs images
Modèle ML	Système	Renvoie la classe et le score d'une image

2.2 Description des cas d'utilisation

ID	Cas d'utilisation	Acteur	Résultat attendu
UC1	Préparer les données	Équipe	Recadrages filtrés, normalisés et découpés en train, validation et test
UC2	Entraîner et comparer les classifieurs	Équipe	Tableau comparatif de SVM, Random Forest et k-NN sur la validation
UC3	Évaluer le modèle final	Équipe, encadrant	Scores sur le test, matrice de confusion, analyse des erreurs
UC4	Classer une nouvelle image	Utilisateur	« photo.jpg → Voiture (92 %) »
UC5	Lancer une démonstration de conduite	Utilisateur, jury	Le véhicule rejoint sa destination en réagissant aux obstacles
UC6	Comparer les classifieurs dans le simulateur	Équipe	Collisions, fautes graves et décisions inadaptées sur plusieurs épisodes
UC7	Consulter les résultats et le rapport	Encadrant, jury	Notebooks commentés, figures et rapport

2.3 Scénarios de fonctionnement

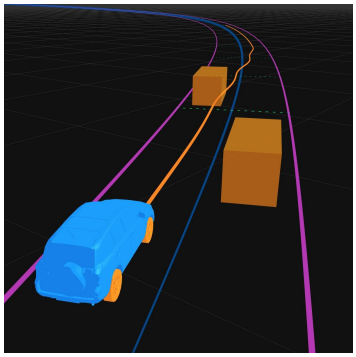
Ces quatre scénarios servent à concevoir les tests et à montrer à l'encadrant ce que le système final devra accomplir. Les images sont des illustrations d'inspiration issues d'autres simulateurs : elles ne montrent pas le nôtre, dont une capture figure en section 4.2.



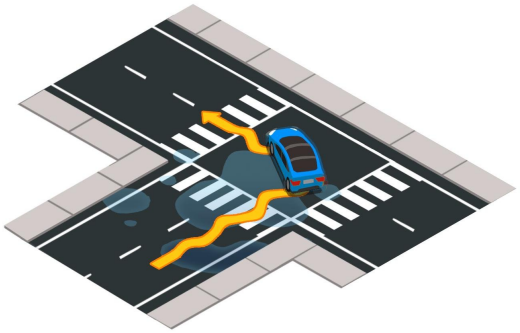
(a) Scénario 1 : route libre.



(b) Scénario 2 : présence d'un piéton.



(c) Scénario 3 : route bloquée.



(d) Scénario 4 : contournement.

Figure 2.1 – Les quatre scénarios de démonstration (illustrations externes, annexe A).

Scénario	Situation	Comportement attendu
1. Route libre (cas nominal)	Aucun obstacle sur l'itinéraire	Le véhicule suit le chemin calculé par Dijkstra jusqu'à la destination
2. Présence d'un piéton	Un objet classé « personne » est sur l'itinéraire, à moins de 35 m	Le véhicule s'arrête, attend que la personne traverse, puis repart sur le même trajet
3. Route bloquée	Un objet classé « voiture », « bus » ou « camion » occupe le tronçon	Le tronçon est marqué bloqué dans le graphe, et Dijkstra cherche un autre chemin
4. Contournement	Un nouveau chemin existe	Le véhicule change de rue, ou fait demi-tour si besoin, puis reprend sa progression

Cas d'erreur. Si le score est trop faible, la classe devient « Inconnu » et le véhicule fait un arrêt prudent. S'il n'existe plus aucun chemin, le véhicule reste arrêté ; au bout de 30 pas, l'épisode est déclaré « bloqué ».

2.4 Diagramme de cas d'utilisation

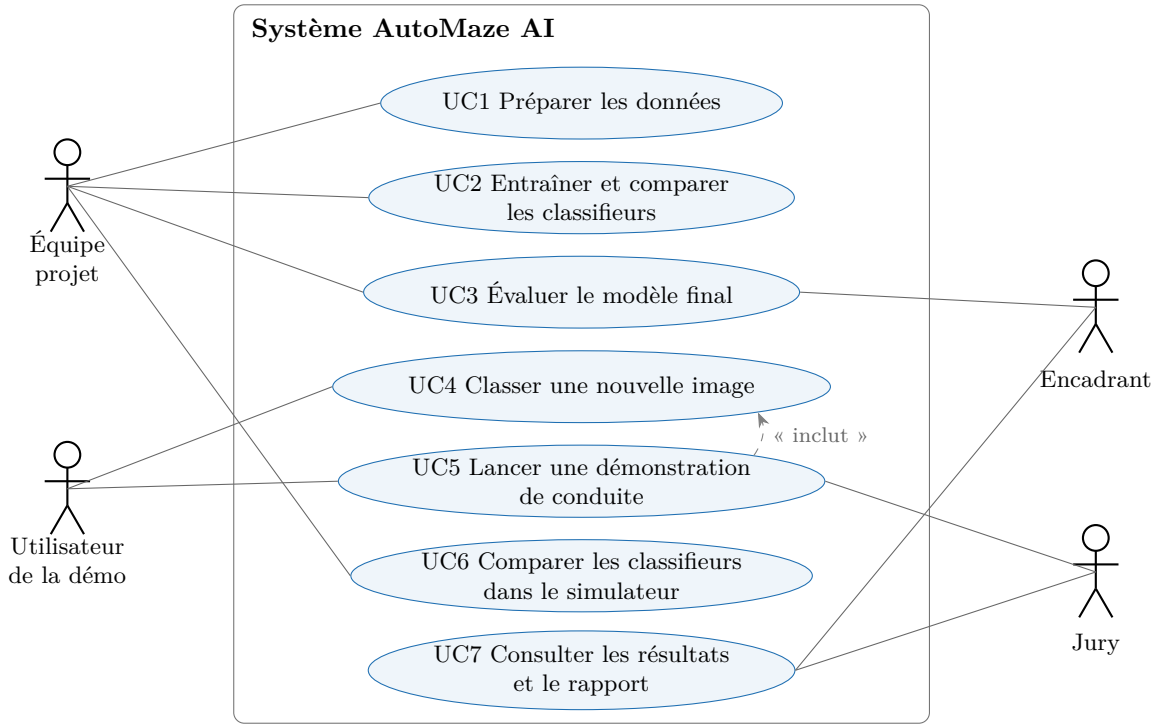


Figure 2.2 – Diagramme de cas d'utilisation.

2.5 Diagramme d'activité

Le diagramme suivant décrit un épisode de conduite, du calcul de l'itinéraire à l'arrivée.

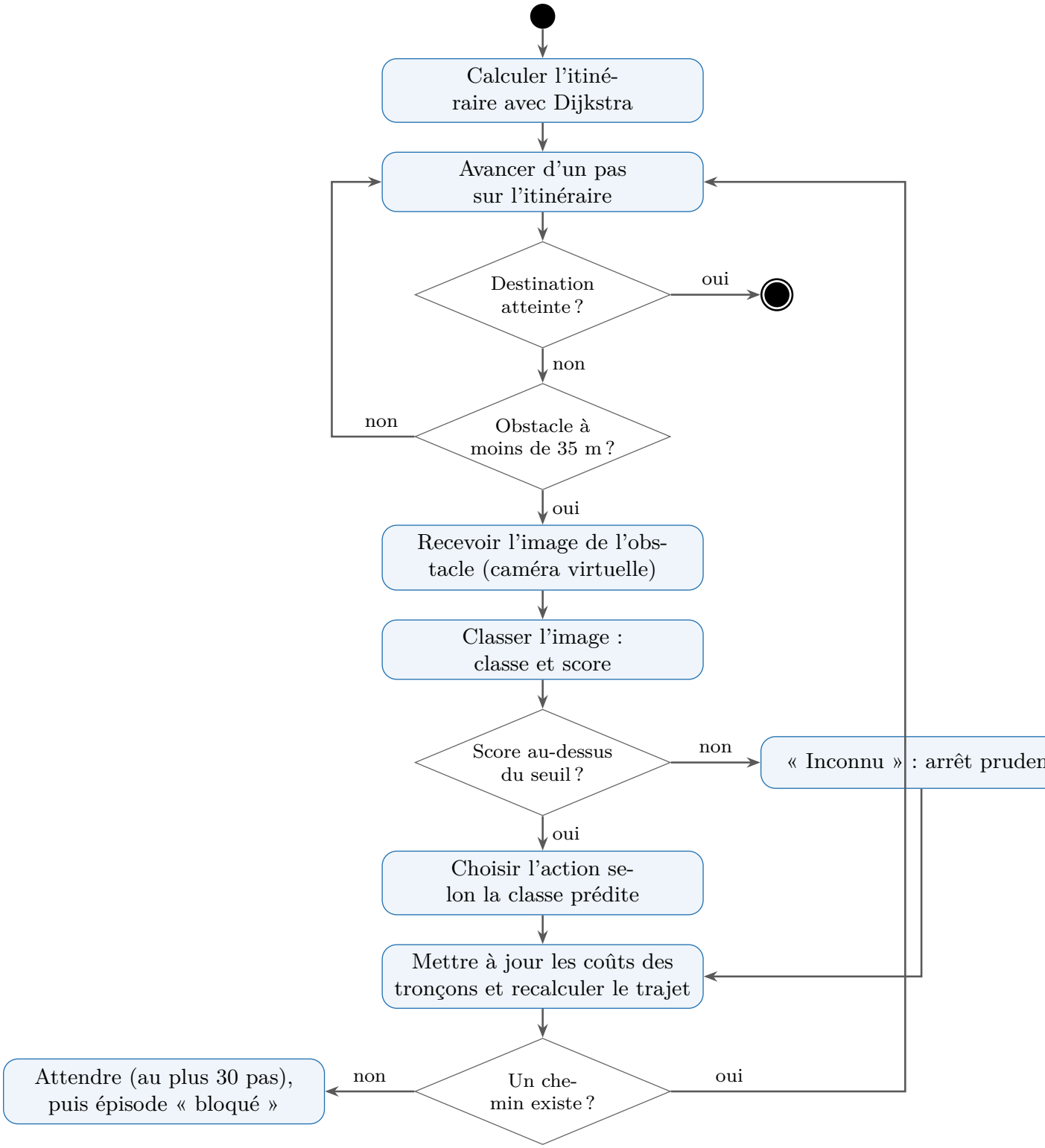


Figure 2.3 – Diagramme d'activité d'un épisode.

2.6 Diagramme de séquence

Ce diagramme montre les échanges entre modules quand le véhicule rencontre un obstacle. La vraie classe de l'objet n'apparaît dans aucun message : le système de décision ne reçoit que la classe prédite.

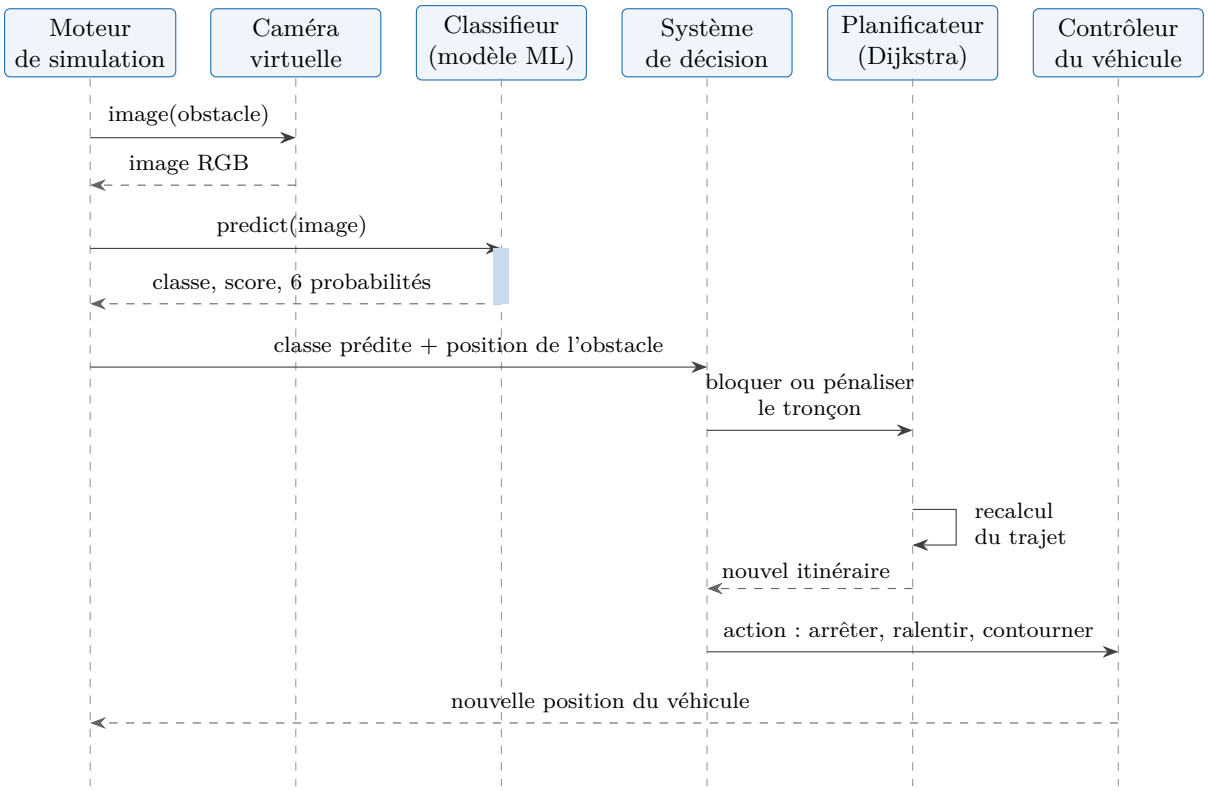


Figure 2.4 – Diagramme de séquence du traitement d'un obstacle.

2.7 Diagramme de composants

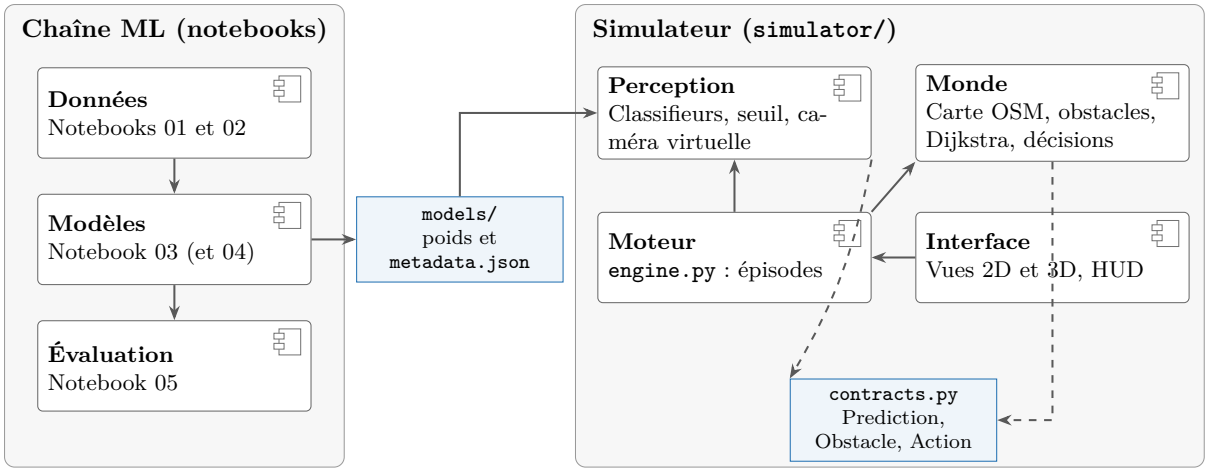


Figure 2.5 – Diagramme de composants. Les flèches pleines indiquent les appels ou les échanges de fichiers ; les flèches pointillées, l'usage des contrats partagés.

2.8 Description des interactions entre modules

De	Vers	Information échangée
Notebooks	Simulateur	Poids du modèle et `metadata.json` : classes, taille d'entrée, ordre des couleurs, normalisation, seuil
Moteur	Caméra virtuelle	Obstacle rencontré ; en retour, une image RGB
Moteur	Classifieur	Image RGB ; en retour, une `Prediction` : classe, score, 6 probabilités, temps de calcul
Moteur	Décision	Classe prédite et tronçon de l'obstacle ; en retour, l'action à appliquer
Décision	Planificateur	Tronçons bloqués ou pénalisés ; en retour, un nouvel itinéraire
Moteur	Contrôleur	Itinéraire, attente ou ralentissement ; en retour, la position du véhicule
Moteur	Interface	État de l'épisode, dernière image, prédiction et décision

Chapitre 3

Étude et préparation des données

3.1 Étude des datasets disponibles

Option	Classes couvertes	Atouts	Limites
COCO 2017	Les 6	Photos réelles, une boîte annotée par objet	Fort déséquilibre, beaucoup de petits objets, recadrages à faire
CIFAR-10 + CIFAR-100	Les 6, en combinant les deux	Téléchargement simple, images déjà centrées	32×32 px seulement, peu de scènes routières
Pascal VOC 2012	5 sur 6	Annotations soignées, taille raisonnable	Pas de classe camion
Jeux Kaggle « véhicules »	Variable	Prêts à l'emploi	Qualité et licences inégales, souvent sans personne ni vélo

3.2 Comparaison et justification du dataset retenu

Nous retenons **COCO 2017**, sous réserve de l'accord de l'encadrant. C'est la seule option qui couvre nos six classes avec de vraies photos, prises dans des conditions variées, et des annotations de référence. Ses boîtes nous permettent de découper chaque objet. Nous n'avons pas à télécharger les 18 Go d'images : seules les photos utiles sont récupérées, grâce à leur adresse.

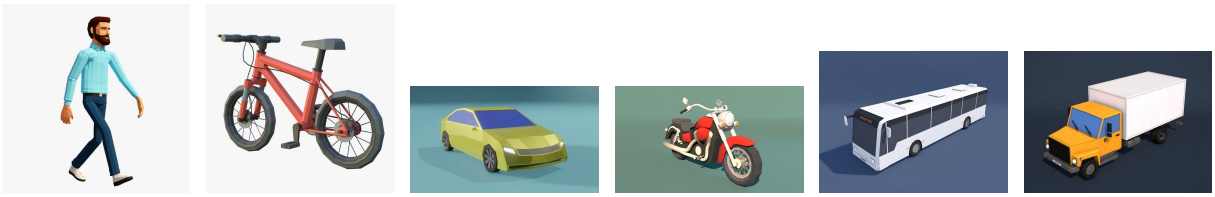
Les annotations sont sous licence CC BY 4.0, et chaque image garde la licence Flickr choisie par son auteur. Un usage scolaire est possible à condition de citer COCO et de ne pas redistribuer les images. CIFAR reste notre solution de secours, avec l'inconvénient d'une résolution très faible.

3.3 Présentation des classes

Classe	Nom dans COCO	Pourquoi elle compte pour un véhicule
Personne	`person`	Usager le plus vulnérable : une erreur sur cette classe est la plus grave
Vélo	`bicycle`	Usager lent et fragile, souvent proche d'une personne. COCO annote le vélo, pas le cycliste
Voiture	`car`	Usager le plus fréquent, souvent à l'arrêt
Moto	`motorcycle`	Petit véhicule rapide, proche visuellement du vélo
Bus	`bus`	Grand véhicule qui masque la visibilité
Camion	`truck`	Grand véhicule ; COCO y range aussi des pick-ups

L'ordre des classes (personne, vélo, voiture, moto, bus, camion) est fixé une fois pour toutes. Il

correspond aux sorties du modèle et à ce qu'attend le simulateur.



Personne | Vélo | Voiture | Moto | Bus | Camion

Figure 3.1 – Les six classes, illustrées par des modèles 3D (sources externes, annexe A). Le modèle n'apprend pas sur ces images, mais sur des photos réelles de COCO (figure 3.3).

3.4 Analyse quantitative des données

L'ensemble `train2017` compte 118 287 images, dont 70 082 contiennent au moins un de nos objets. Leur résolution médiane est de 640 × 480 px.

Classe	Objets annotés	Hors foules, plus petit côté d'au moins 32 px
Personne	262 465	150 185
Voiture	43 867	16 799
Camion	9 973	7 071
Moto	8 725	6 742
Vélo	7 113	4 505
Bus	6 069	5 194

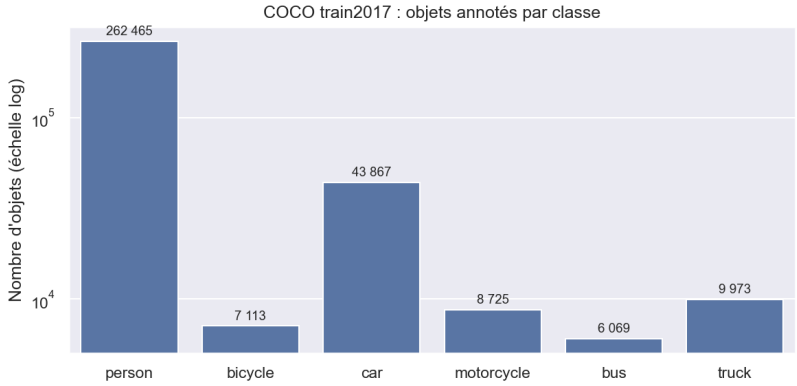


Figure 3.2 – Répartition des objets par classe dans COCO `train2017`, avant filtrage.

Le déséquilibre est fort : on compte 43,2 personnes pour un bus. Sans précaution, un modèle pourrait viser une bonne accuracy en répondant presque toujours « personne ». Nous plafonnons donc le nombre d'objets par classe, et nous jugeons les modèles au F1 macro.

3.5 Analyse qualitative des images

L'observation d'exemples tirés au hasard a montré :

— des objets coupés par le bord de la photo ou cachés par un autre objet ;

— des images floues ou très petites, surtout pour les voitures lointaines ;

— plusieurs objets dans un même recadrage, comme un motard sur sa moto ;

— une classe « camion » hétérogène, qui contient aussi des pick-ups et des camionnettes, proches d'une voiture ;

— des photos prises par des particuliers, et non depuis un véhicule : un biais à garder en tête pour l'usage routier.

3.6 Nettoyage des données

— **Foules écartées :** une boîte marquée `iscrowd` entoure un groupe et non un seul usager.

— **Petits objets écartés :** sous 32 px de côté, l'objet devient trop flou une fois agrandi. À ce seuil, la classe la plus rare garde 4 505 objets ; à 64 px, il n'en resterait que 2 693.

— **Plafond de 2 500 objets par classe :** les classes sont équilibrées, et l'entraînement reste faisable sans GPU.

— **Téléchargements contrôlés :** une photo inaccessible est notée, sans arrêter la préparation. Sur 15 000 objets prévus, 14 990 ont été récupérés : 8 téléchargements ont expiré.

3.7 Redimensionnement et normalisation

Chaque objet est découpé dans un **carré** centré sur sa boîte, avec une marge de 10 %, puis réduit à $\mathbf{64 \times 64}$ **px**. Le carré évite de déformer les silhouettes : une personne est haute et fine, un bus est large. Quand le carré dépasse de la photo, les pixels manquants sont complétés par reflet plutôt que par une bande noire, que le modèle pourrait apprendre à tort.

Chaque canal de couleur est ensuite normalisé avec la moyenne et l'écart-type du jeu d'entraînement. Les valeurs mesurées sont [0,4057 ; 0,3894 ; 0,3748] pour la moyenne et [0,2609 ; 0,2554 ; 0,2571] pour l'écart-type. Le simulateur applique exactement ces valeurs.

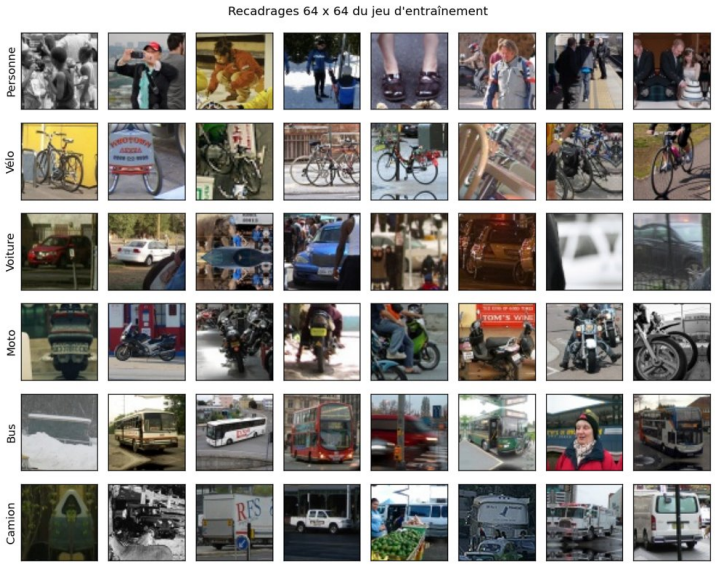


Figure 3.3 – Recadrages 64 × 64 du jeu d'entraînement, une ligne par classe (photos COCO).

3.8 Séparation entraînement, validation et test

Classe	Entraînement (70 %)	Validation (15 %)	Test (15 %)
Personne	1 750	375	375
Vélo	1 748	373	375
Voiture	1 750	375	374
Moto	1 750	374	375
Bus	1 748	374	375
Camion	1 749	375	375

Le découpage est **stratifié**, c'est-à-dire qu'il garde les mêmes proportions de classes dans chaque jeu. Il se fait aussi **par photo** : deux objets d'une même photo partagent le même éclairage et le même décor, et les répartir entre le train et le test gonflerait le score. Un contrôle vérifie qu'aucune des 10 672 photos n'apparaît dans deux jeux. Le découpage est enregistré dans un fichier commun, pour que toute l'équipe utilise exactement les mêmes jeux.

Chapitre 4

Architecture technique et conception des modules

4.1 Architecture globale d'AutoMaze AI

Le principe est le suivant. Le simulateur connaît la position de chaque objet de son environnement. Le modèle de Machine Learning analyse l'image fournie par la caméra virtuelle et en déduit la nature de l'objet. Le système combine ensuite cette classe prédite avec la position connue. Le planificateur calcule le chemin vers la destination et le recalcule si un obstacle rend le trajet impraticable. Enfin, le contrôleur applique les décisions au véhicule : continuer, ralentir, s'arrêter ou changer de direction.

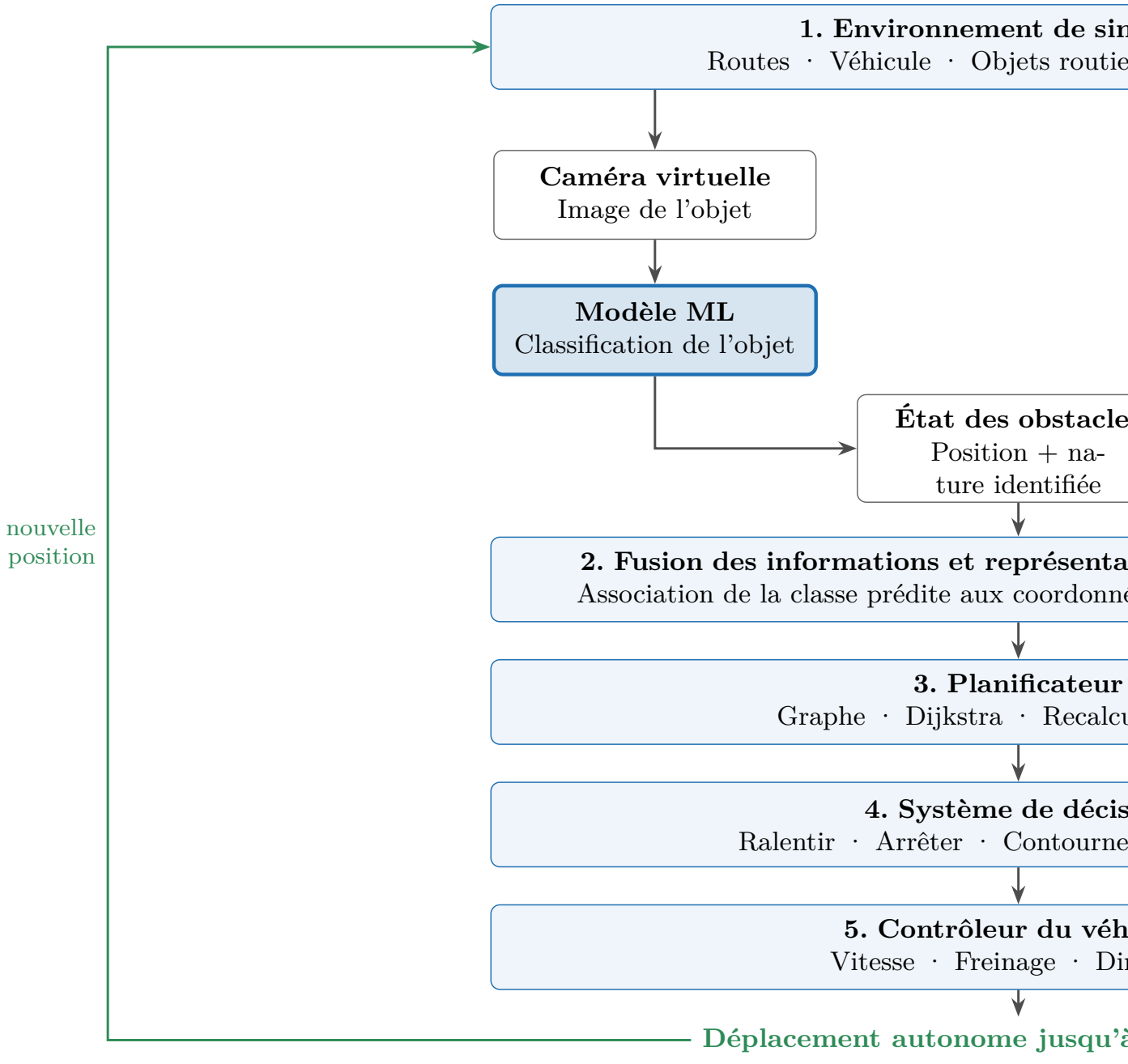


Figure 4.1 – Architecture fonctionnelle globale. Le modèle ML (en bleu foncé) est le cœur du projet ; les autres étages servent à la démonstration.

4.2 Environnement de simulation

Notre environnement reproduit une partie réelle de Casablanca, du Twin Center (Maârif) à la place Mohammed V. Le réseau routier vient d'OpenStreetMap, avec ses sens uniques. Le véhicule roule à 8 m/s, soit environ 29 km/h, et le temps avance par pas d'une seconde. Chaque épisode place 6 obstacles sur l'itinéraire. Trois vues sont proposées : une carte 2D, une vue conducteur et une vue 3D.

Les simulateurs de conduite professionnels offrent des environnements 3D détaillés (figure 4.2). Le nôtre reste volontairement plus simple, car il ne sert qu'à montrer l'usage du modèle.



Figure 4.2 – Exemples d'environnements virtuels de conduite, qui nous ont servi d'inspiration (sources externes, annexe A). Ce ne sont pas des captures de notre simulateur.

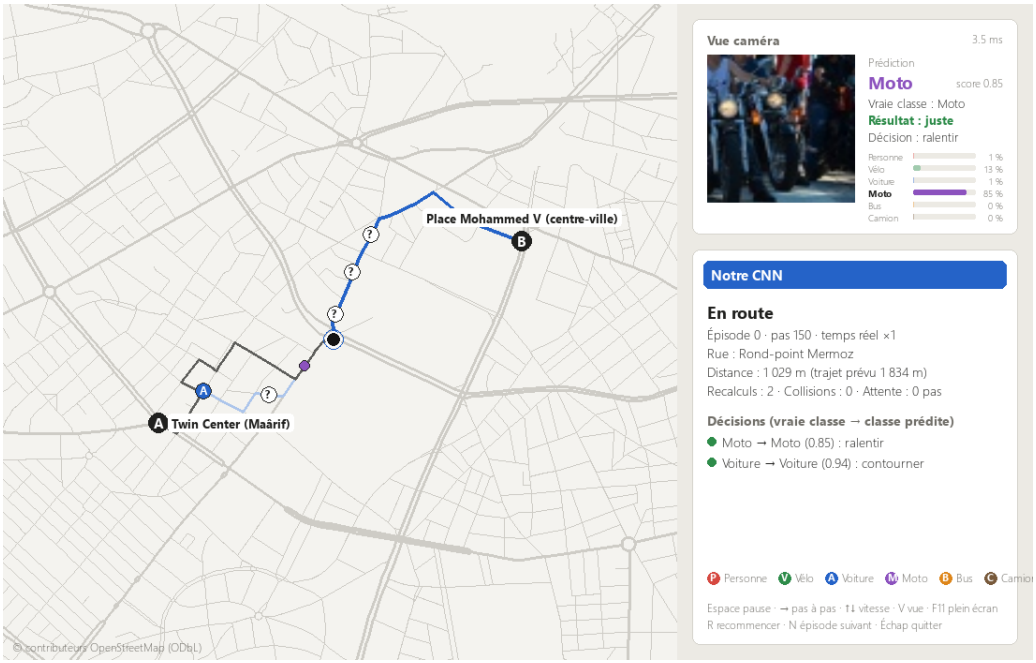


Figure 4.3 – Notre simulateur, vue carte : itinéraire calculé par Dijkstra, obstacles (?) encore inconnus du véhicule, et à droite l'image donnée au classifieur avec sa prédiction. Carte : © contributeurs OpenStreetMap (ODbL).

4.3 Acquisition des images par caméra virtuelle

Quand un obstacle se trouve sur l'itinéraire à moins de 35 m, la caméra virtuelle fournit son image au modèle.

Un choix de conception important. Si la caméra filmait les objets 3D du simulateur, le modèle verrait des images de synthèse très différentes des photos sur lesquelles il a appris. Ses performances chuteraient sans que ce soit sa faute. La caméra fournit donc une *vraie photo* de la classe de l'obstacle, tirée du jeu de test, que le modèle n'a jamais vue. Les modèles 3D ne servent qu'à l'affichage. Une même graine donne les mêmes images à tous les classifieurs comparés, et l'image change d'un épisode à l'autre.

4.4 Module de classification par Machine Learning

Le module reçoit l'image RGB d'un objet, quelle que soit sa taille. Il applique lui-même le prétraitement : mise au carré, réduction à 64 × 64 px, normalisation. Il renvoie ensuite :

— la classe prédite, parmi les six ;

— un score de confiance, le plus grand des six scores ;

— les six scores, dont la somme vaut 1 ;

— le temps de calcul réellement mesuré.

Exemple d'affichage : « Image → Voiture (92 %) ». Ce 92 % est un exemple d'affichage, pas une performance mesurée. Il s'agit d'un score, pas d'une probabilité calibrée : le modèle n'a pas forcément raison 92 fois sur 100.

Seuil « Inconnu ». Sous un certain score, la classe devient « Inconnu » et le véhicule fait un arrêt prudent. Le seuil sera choisi sur le jeu de validation, puis enregistré avec le modèle.

Interchangeabilité. Trois classifieurs respectent la même interface : un mode « simulation » qui teste la logique sans modèle, notre modèle, et un éventuel modèle de référence préentraîné, utilisé seulement comme point de comparaison si l'encadrant l'accepte.

4.5 Module de localisation et association des objets

Le simulateur décrit chaque obstacle par un identifiant et une position sur la carte.

Identifiant	Position	Classe prédite	Vraie classe (cachée)
OBJ-01	Tronçon A → B, à 45 m de A	Voiture	Voiture
OBJ-02	Tronçon C → D, à 12 m de C	Personne	Personne
OBJ-03	Tronçon E → F, à 30 m de E	Voiture	Camion

Association. Chaque image est produite *pour* un obstacle donné. La prédiction est donc rattachée directement à l'objet dont provient l'image, sans projection entre l'image et le monde virtuel. Ce choix est simple et sûr, parce qu'il y a un seul objet par image.

Vérité terrain et perception, bien séparées :

— **Vérité terrain :** la vraie classe et la position. Elles servent à construire les scénarios, à faire agir l'obstacle selon sa vraie nature (une personne finit de traverser, un camion reste immobile) et à mesurer les erreurs.

— **Perception :** la classe prédite par notre modèle. C'est la seule que reçoit le système de décision. Une erreur du modèle a donc de vraies conséquences, comme dans l'exemple OBJ-03, qui peut mener à une collision.

4.6 Représentation de l'environnement et des obstacles

Le réseau routier est un graphe orienté : les nœuds sont les intersections, et les arêtes les tronçons de rue. Chaque tronçon a un coût égal à sa longueur en mètres, multipliée par une éventuelle pénalité. Un obstacle peut modifier ce graphe de trois façons :

— **bloquer** un tronçon, pour toujours ou pendant quelques pas ;

— le **pénaliser**, c'est-à-dire multiplier son coût ;

— pénaliser aussi les **tronçons voisins**, pour un gros véhicule.

4.7 Algorithme de planification Dijkstra

L'algorithme de Dijkstra, écrit à la main, calcule le chemin de coût minimal entre la position actuelle et la destination. Après chaque modification du graphe, il est relancé depuis la position du véhicule. Si l'obstacle se trouve sur le tronçon actuel, le véhicule peut faire demi-tour. Les sens uniques sont respectés.

Le chemin le plus court n'est pas toujours le plus rapide ni le plus sûr. Nous partons des distances comme coûts, puis nous ajoutons des pénalités pour les passages moins favorables, par exemple une rue où roule un vélo.

4.8 Système de décision

La décision dépend de la classe prédite. Elle ne se déclenche que pour un obstacle situé sur l'itinéraire, devant le véhicule, à moins de 35 m.

Situation	**Décision**	**Effet**
Aucun obstacle sur la trajectoire	Continuer	Le véhicule suit son itinéraire
Personne	S'arrêter et attendre	Attente de 5 pas, puis reprise du même trajet
Vélo ou moto	Ralentir	Tronçon 3 fois plus coûteux ; vitesse divisée par 3 s'il reste le meilleur chemin
Voiture	Contourner	Tronçon bloqué, puis nouvel itinéraire
Bus ou camion	Contourner largement	Tronçon bloqué, rues voisines 10 fois plus coûteuses, nouvel itinéraire
Score trop faible (« Inconnu »)	Arrêt prudent	Attente de 2 pas, tronçon bloqué, nouvel itinéraire
Aucun chemin possible	Rester arrêté	Nouvel essai à chaque pas, épisode « bloqué » après 30 pas
Voie dégagée (fin de l'attente)	Reprendre	Le véhicule repart sur son itinéraire

Limite assumée. Dans cette première version, l'action dépend seulement de la classe. La distance et la position de l'objet ne servent qu'à déclencher la décision. Une règle plus réaliste tiendrait aussi compte de la distance, de la trajectoire et de la vitesse relative. Par exemple, un piéton sur le trottoir ne demande pas le même arrêt qu'un piéton sur la chaussée. Nous proposons d'ajouter ce raffinement si le temps le permet.

4.9 Contrôle du véhicule virtuel

Le contrôleur reste simplifié : il ne modélise ni les forces ni l'adhérence. À chaque pas, il applique l'une des commandes suivantes :

— **avancer** de 8 m le long de l'itinéraire ;

— **ralentir**, en divisant la vitesse par 3 sur un tronçon pénalisé ;

— **s'arrêter** pendant le nombre de pas décidé ;

— **changer de direction** en suivant le nouvel itinéraire, ou faire demi-tour si nécessaire ;

— **reprendre** le déplacement quand la voie est libre.

Une collision est comptée quand le véhicule atteint un obstacle encore présent. Cela arrive par exemple si un camion immobile a été pris pour un vélo : le véhicule ralentit au lieu de contourner.

4.10 Communication et intégration des modules

Point d'échange	**Rôle**
`simulator/contracts.py`	Seul fichier partagé entre les modules : définit la prédiction, l'obstacle, les actions et l'interface commune des classifieurs
`models/metadata.json`	Décrit le modèle : classes dans l'ordre, taille d'entrée, ordre des couleurs, normalisation, seuil
Poids du modèle	Chargés par le simulateur, qui ne réentraîne jamais le modèle
Journal des rencontres	Une ligne par obstacle (classe réelle, classe prédite, score, action, temps), utilisée pour mesurer les erreurs

Le simulateur vérifie au chargement que l'ordre des classes est le bon. Sinon, un bus pourrait devenir une personne sans aucun message d'erreur. Un test de concordance compare aussi les sorties du simulateur et celles du notebook sur les mêmes images, pour s'assurer que le prétraitement est identique.

Chapitre 5

Méthodologie expérimentale et évaluation

5.1 Extraction des caractéristiques

SVM, Random Forest et k-NN ne lisent pas directement une image : il faut d'abord la résumer en une liste de nombres, le *descripteur*. Nous utilisons **HOG** (*Histogram of Oriented Gradients*), qui décrit l'orientation des contours :

1. l'image 64×64 est convertie en niveaux de gris ;
2. elle est découpée en cellules de 8×8 px, et chaque cellule compte ses contours selon 9 orientations ;
3. les cellules sont regroupées en blocs de 2×2, normalisés pour compenser les différences d'éclairage.

Un bloc de 16 px, décalé de 8 px à chaque fois, prend 7 positions par côté, ce qui donne $7 \times 7 = 49$ blocs de 4×9 valeurs : $49 \times 4 \times 9 = 1\,764$ nombres par image. Nous testerons aussi des cellules de 4 px, plus fines mais beaucoup plus nombreuses. HOG ignore la couleur : c'est une limite connue, qui pèsera surtout sur des classes de formes proches, comme le bus et le camion.

5.2 Présentation des algorithmes SVM, Random Forest et k-NN

Algorithme	Principe	Points forts et limites	Réglages testés
SVM	Cherche la frontière qui sépare les classes avec la plus grande marge. Avec un noyau RBF, cette frontière peut être courbe	Efficace en grande dimension ; lent à entraîner quand les exemples sont nombreux	Linéaire ou RBF ; paramètre C
Random Forest	Fait voter des centaines d'arbres de décision, chacun entraîné sur un tirage différent des données	Rapide et robuste ; chaque question d'un arbre ne regarde qu'une valeur à la fois	Nombre d'arbres, profondeur
k-NN	Donne la classe majoritaire parmi les k images d'entraînement les plus proches	Simple, sans vrai entraînement ; les distances perdent leur sens en grande dimension	k ; réduction par PCA (50 à 200 dimensions)

Le SVM et le k-NN sont précédés d'une mise à l'échelle des valeurs, car ils y sont sensibles. Le Random Forest n'en a pas besoin.

Extensions, sous réserve de l'accord de l'encadrant. En plus des trois algorithmes du sujet, nous proposons XGBoost, une autre famille d'arbres construits l'un après l'autre, et un petit réseau convolutif (CNN) entraîné de zéro. Ces deux extensions sont comparées de la même façon, sans remplacer les trois modèles demandés.

5.3 Protocole expérimental

1. **Même point de départ :** tous les modèles utilisent les mêmes recadrages, le même découpage et les mêmes descripteurs.
2. **Réglages sur la validation :** chaque combinaison de réglages est entraînée sur le train, puis mesurée sur la validation.
3. **Une seule chose change à la fois :** chaque essai est noté avec ses réglages, son score et son temps d'entraînement.
4. **Test ouvert une seule fois :** les modèles retenus sont mesurés sur le test à la fin. Plus aucun réglage n'est permis après.
5. **Reproductibilité :** les graines aléatoires sont fixées, et les notebooks s'exécutent de bout en bout.

Le test reste fermé jusqu'à la fin, car un modèle réglé en regardant le test aurait un score trop optimiste. C'est comme un élève qui connaîtrait les questions de l'examen.

5.4 Métriques d'évaluation

Métrique	**Ce qu'elle mesure**
Accuracy	Part de bonnes réponses sur toutes les images
Précision (par classe)	Quand le modèle répond « camion », part de vrais camions
Rappel (par classe)	Parmi les vrais camions, part que le modèle retrouve
F1 (par classe)	Moyenne harmonique de la précision et du rappel : $F1 = \frac{2 \times \text{précision} \times \text{rappel}}{\text{précision} + \text{rappel}}$
F1 macro	Moyenne des F1 des 6 classes, chacune avec le même poids : **métrique principale**
Rappel « personne »	Part des personnes reconnues : l'erreur la plus grave pour un véhicule
Matrice de confusion	Croise vraies classes et classes prédites, pour voir quelles classes se confondent
Temps d'inférence	Temps de calcul d'une prédiction sur CPU

5.5 Méthode de comparaison des modèles

— **Mêmes données :** les modèles sont comparés sur les mêmes images, dans le même ordre.

— **Scores planchers :** un modèle qui répond toujours la même classe, ou qui tire au hasard selon les fréquences, donne la valeur à dépasser.

— **Analyse des erreurs :** au-delà du score global, nous comparons les matrices de confusion et regardons des images mal classées tirées au hasard.

— **Incertitude :** avec environ 2 250 images de test, une accuracy n'est connue qu'à environ $\pm$ 2 points près. Un écart plus petit ne permet pas de départager deux modèles.

5.6 Critères de sélection du modèle final

1. le meilleur **F1 macro sur la validation** ;
2. à score proche, le meilleur **rappel « personne »** ;
3. puis un **temps d'inférence** compatible avec le simulateur (quelques millisecondes) ;
4. enfin, le modèle le plus simple et le plus stable d'un essai à l'autre.

Le modèle retenu est exporté avec ses métadonnées. Son seuil « Inconnu » est choisi sur la validation : c'est le plus haut seuil qui laisse au moins 90 % des images classées.

5.7 Protocole de validation dans le simulateur

Chaque classifieur parcourt exactement les mêmes épisodes : même carte, même itinéraire, mêmes obstacles et mêmes images. Le protocole prévoit 50 épisodes, avec une graine fixée.

Mesure	**Définition**
Épisodes arrivés	Le véhicule a atteint la destination
Collisions	Le véhicule a atteint un obstacle encore présent
Obstacles bien classés	Classe prédite égale à la vraie classe
Fautes graves	Personne classée autrement que « personne », « Inconnu » compris
Décisions inadaptées	Action différente de celle que demandait la vraie classe
Détour et recalculs	Distance parcourue rapportée au trajet prévu, nombre de nouveaux calculs
Temps par décision	Temps moyen d'une prédiction, mesuré sur CPU

Le mode « simulation » n'est pas un modèle : son taux d'erreur est réglé à la main. Il vérifie seulement que la logique du simulateur fonctionne, et ses chiffres ne se comparent pas à ceux d'un vrai classifieur.

Chapitre 6

Étude de faisabilité et validation Go/noGo

6.1 Faisabilité technique

Chaque brique technique du projet repose sur des méthodes connues et des bibliothèques éprouvées :

— **Données :** COCO fournit des boîtes annotées pour nos six classes, ce qui permet de découper les objets automatiquement.

— **Classification :** HOG, SVM, Random Forest et k-NN sont disponibles dans OpenCV et scikit-learn.

— **Planification :** Dijkstra sur un graphe routier se programme en quelques dizaines de lignes, et OpenStreetMap fournit la carte.

— **Intégration :** le simulateur et le modèle ne communiquent que par un fichier de métadonnées et une interface commune, ce qui limite les risques d'incompatibilité.

La principale difficulté technique se trouve dans les données : classes déséquilibrées, petits objets et étiquettes parfois discutables. Le chapitre 3 explique comment nous la traitons.

6.2 Faisabilité matérielle et logicielle

Aucun membre n'a de GPU. Les premiers essais montrent que ce n'est pas bloquant : les 21 essais de modèles classiques ont demandé 22 minutes d'entraînement au total sur un processeur ordinaire. Google Colab reste disponible en appui, avec son GPU gratuit.

Besoin	**Outil retenu**	**Pourquoi**
Notebooks et calcul	Jupyter, VS Code, Google Colab	Même environnement pour les trois membres, GPU gratuit si besoin
Partage	GitHub pour le code, Google Drive pour les données	Pas de fichiers lourds dans Git
Données et figures	NumPy, pandas, Matplotlib, seaborn	Exploration et graphiques
Images	OpenCV	Recadrage, redimensionnement, descripteurs HOG
Classifieurs	scikit-learn	SVM, Random Forest, k-NN et métriques
Extensions	XGBoost, PyTorch	Boosting et CNN de zéro, si accord
Carte	OSMnx, OpenStreetMap	Vraies rues de Casablanca
Affichage	pygame, Panda3D	Vues 2D et 3D, sans GPU dédié
Tests	pytest	Vérifier le code à chaque modification

6.3 Identification des risques

Risque	Niveau	Parade
Dataset refusé par l'encadrant	Moyen	CIFAR en solution de secours ; la chaîne de traitement reste la même
Scores trop faibles sur certaines classes	Élevé	Analyse des confusions ; extension CNN qui utilise la couleur
Fuite de données entre les jeux	Moyen	Découpage par photo, contrôle automatique, test ouvert une seule fois
Prétraitement différent entre notebook et simulateur	Moyen	Paramètres lus dans les métadonnées, test de concordance
Écart entre images d'entraînement et simulateur	Moyen	La caméra virtuelle fournit de vraies photos, pas des rendus 3D (section 4.3)
Intégration qui prend le pas sur le Machine Learning	Moyen	Le modèle reste prioritaire ; le simulateur s'arrête à sa version actuelle si besoin
Manque de temps avant la soutenance	Moyen	Modèle figé le 9 décembre, une semaine avant la soutenance

6.4 Preuves de faisabilité disponibles

Ces éléments montrent que le projet est réalisable. Ils ne signifient pas que tout est terminé : les scores ci-dessous sont mesurés sur la *validation*, et le jeu de test reste réservé à l'évaluation finale.

Preuve	Ce qu'elle démontre	État au 6 novembre
Étude du dataset	Les données sont disponibles et adaptées	COCO étudié, licences vérifiées (chapitre 3)
Notebook d'exploration	Les classes et leur distribution sont connues	Effectifs, tailles et exemples mesurés
Notebook de prétraitement	Les images peuvent être préparées	14 990 recadrages, découpage sans photo partagée
Premier classifieur	Le principe de classification fonctionne	SVM à noyau RBF : F1 macro de 0,591 en validation, contre 0,171 pour le hasard
Comparaison préliminaire	Les modèles peuvent être évalués	SVM, Random Forest, k-NN et XGBoost comparés (figure 6.1)
Architecture globale	Les modules et leurs échanges sont définis	Chapitres 2 et 4
Prototype de planification	Dijkstra trouve un chemin et en cherche un autre	Implémenté sur la carte de Casablanca, couvert par des tests automatiques
Scénario de simulation	L'intégration est techniquement plausible	Le simulateur charge un modèle et déroule les quatre scénarios (figure 4.3)

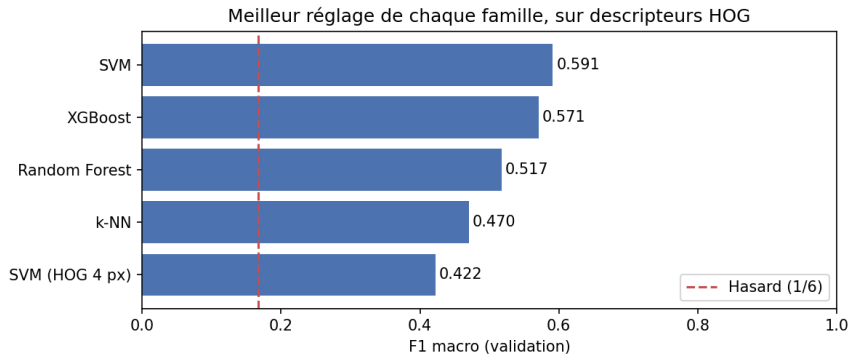


Figure 6.1 – Comparaison préliminaire sur la validation : meilleur réglage de chaque famille, sur descripteurs HOG. La ligne rouge indique le hasard (1/6).

Ce que montrent ces premiers résultats :

- tous les modèles dépassent nettement le hasard ;
- le SVM à noyau RBF arrive en tête (0,591), devant XGBoost (0,571), le Random Forest (0,517) et le k-NN (0,470) ;
- les confusions principales touchent le camion, souvent pris pour une voiture ou un bus, et le vélo, pris pour une personne ou une moto ;
- un premier CNN entraîné de zéro atteint 0,726 en validation. Cette extension reste soumise à l'accord de l'encadrant.

6.5 Planning de réalisation

Le projet dure six semaines, du 4 novembre au 16 décembre 2026. Les dates des séances et du gel du modèle sont fixes ; les durées des travaux restent des estimations. Si une étape prend du retard, ce sont les étapes suivantes qui se décalent, jamais les séances.

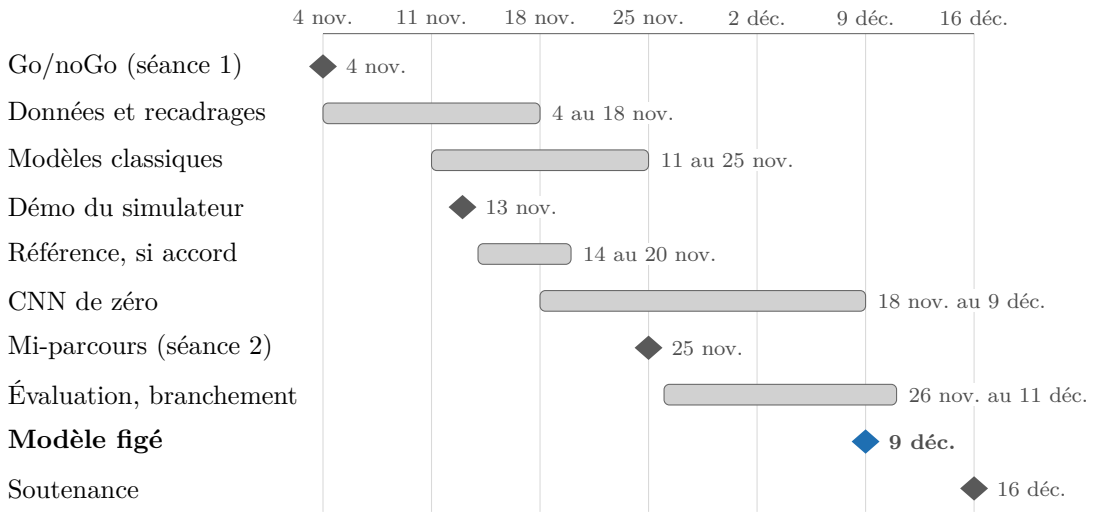


Figure 6.2 – Calendrier prévisionnel : le modèle est figé le 9 décembre, une semaine avant la soutenance.

La figure 6.3 donne la répartition des efforts que nous visons. Le Machine Learning et les données

en prennent la plus grande part, car ils forment le cœur du sujet.

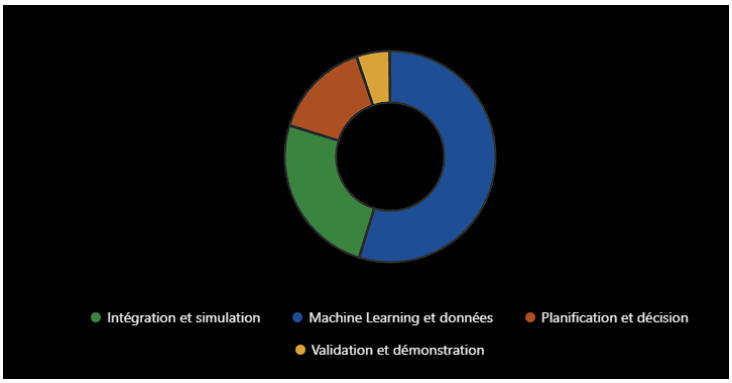


Figure 6.3 – Répartition indicative des efforts : c'est une proposition de répartition du travail, pas une mesure du temps déjà passé.

6.6 Critères de décision Go/noGo

Critère	Constat	Statut
Un dataset exploitable existe	COCO couvre les 6 classes ; données préparées	Rempli
Une méthodologie réaliste existe	Protocole fixé, test réservé, métriques définies	Rempli
La classification fonctionne	Tous les modèles dépassent nettement le hasard	Rempli
L'usage des prédictions est défini	Règles de décision et association prédiction–objet décrites	Rempli
L'architecture est cohérente	Modules, échanges et diagrammes définis	Rempli
Les moyens suffisent	Outils gratuits, entraînement possible sans GPU	Rempli
Les choix sont validés par l'encadrant	Trois points restent à confirmer	À valider

Décision demandée : Go. Pour avancer dans de bonnes conditions, nous avons besoin de l'avis de notre encadrant sur trois points :

- ☐ **Jeu de données :** COCO 2017 recadré convient-il, avec CIFAR comme solution de secours ?
- ☐ **Périmètre :** acceptez-vous XGBoost, le CNN entraîné de zéro et le simulateur comme ajouts au sujet, l'évaluation restant centrée sur SVM, Random Forest et k-NN ?
- ☐ **Modèle de référence :** pouvons-nous utiliser un ResNet-18 préentraîné uniquement comme point de comparaison, bien séparé de notre modèle ? Sans accord, nous le retirons, et le reste du plan ne change pas.

Conclusion générale

Synthèse des choix. AutoMaze AI classe six usagers de la route à partir de l'image d'un objet. Nous avons retenu COCO 2017, des recadrages carrés de 64 × 64 px et un découpage par photo pour éviter toute fuite de données. Les trois classifieurs du sujet, SVM, Random Forest et k-NN, sont comparés sur des descripteurs HOG, au F1 macro. Les prédictions du modèle guident ensuite un véhicule virtuel qui roule dans les rues de Casablanca : la position des objets vient du simulateur, et leur nature vient uniquement du modèle.

Bilan de faisabilité. Les données sont disponibles et préparées, et les premiers classifieurs dépassent nettement le hasard. L'architecture relie la classification d'un objet à une décision de conduite, et les moyens nécessaires sont gratuits. Le projet nous semble donc réalisable dans les délais, sous réserve de l'accord de notre encadrant sur les trois points de la section 6.6.

Perspectives. Une fois ces choix validés, nous terminerons le réglage des modèles sur la validation, puis nous mesurerons le modèle final sur le test et sur de nouvelles images. Si le temps le permet, nous affinerons les règles de décision en tenant compte de la distance et de la position des objets, et nous étudierons l'apport d'un CNN entraîné de zéro.

Bibliographie

[1] T.-Y. Lin, M. Maire, S. Belongie et al., *Microsoft COCO : Common Objects in Context*, European Conference on Computer Vision (ECCV), 2014. https://cocodataset.org

[2] N. Dalal et B. Triggs, *Histograms of Oriented Gradients for Human Detection*, IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2005.

[3] C. Cortes et V. Vapnik, *Support-Vector Networks*, Machine Learning, vol. 20, 1995.

[4] L. Breiman, *Random Forests*, Machine Learning, vol. 45, 2001.

[5] T. Cover et P. Hart, *Nearest Neighbor Pattern Classification*, IEEE Transactions on Information Theory, vol. 13, 1967.

[6] T. Chen et C. Guestrin, *XGBoost : A Scalable Tree Boosting System*, ACM SIGKDD, 2016.

[7] R. Girshick, J. Donahue, T. Darrell et J. Malik, *Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation*, CVPR, 2014.

[8] E. W. Dijkstra, *A Note on Two Problems in Connexion with Graphs*, Numerische Mathematik, vol. 1, 1959.

[9] Contributeurs OpenStreetMap, données sous licence ODbL. https://www.openstreetmap.org

[10] F. Pedregosa et al., *Scikit-learn : Machine Learning in Python*, Journal of Machine Learning Research, vol. 12, 2011.

[11] A. Paszke et al., *PyTorch : An Imperative Style, High-Performance Deep Learning Library*, NeurIPS, 2019.

Annexe A

Sources des illustrations

Les figures produites par nos notebooks et notre simulateur sont les nôtres. Les autres illustrations viennent de sources externes et servent seulement à illustrer le propos.

Illustration	**Source**
Scénarios 1 à 4	**[Source et licence à indiquer]**
Environnements virtuels (figure 4.2)	**[Source et licence à indiquer]**
Détection par R-CNN (figure 1.1)	**[Source et licence à indiquer]**
Modèles 3D des classes	Captures de sites de modèles 3D, dont fab.com (moto) et turbosquid.com (voiture) ; **[auteurs à indiquer]**
Flux de développement, répartition des efforts	Schémas réalisés pour le projet
Statistiques, recadrages et comparaison des modèles	Nos notebooks, à partir de COCO 2017
Capture du simulateur	Notre simulateur ; carte © contributeurs OpenStreetMap (ODbL)

Annexe B

Usage de l'intelligence artificielle

Le simulateur et la structure des notebooks ont été préparés avec l'aide de Claude Code, et la rédaction de ce dossier a aussi été assistée par cet outil. Les résultats préliminaires de la section 6.4 et les chiffres du chapitre 3 viennent de notebooks complétés et exécutés par Claude Code sur la branche `correction` du dépôt, à la demande de l'équipe. Nous les présentons comme des preuves de faisabilité. Leur place dans le travail noté reste à définir avec notre encadrant.